Enunciado Proyecto

SmartCity Analytics

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE-3191 Base de Datos
II Semestre 2026



Grupos máximo de 4 personas
Fecha Entrega: 25 noviembre 2026

1. Introducción

Las ciudades modernas generan continuamente grandes volúmenes de información provenientes de sensores, dispositivos IoT, estaciones meteorológicas, sistemas de transporte, sensores ambientales, medidores inteligentes de energía y plataformas de monitoreo urbano.

La administración eficiente de esta información requiere el uso de tecnologías capaces de gestionar datos estructurados, semiestructurados y masivos, permitiendo su almacenamiento, consulta, integración y análisis en tiempo real e histórico.

Se propone desarrollar una arquitectura integral para la gestión, almacenamiento, procesamiento y análisis de datos urbanos, utilizando diferentes tecnologías de bases de datos y procesamiento masivo.

El proyecto simulará el ecosistema de una Ciudad Inteligente (Smart City) en la cual diferentes dispositivos IoT generan continuamente información sobre el comportamiento de la ciudad.

Los estudiantes deberán demostrar que no existe una única tecnología adecuada para todos los tipos de datos y necesidades, sino que cada plataforma tiene características, ventajas y dominios de aplicación particulares.

El objetivo de este proyecto es diseñar, implementar y demostrar una arquitectura de datos que permita:

- Almacenar de forma eficiente datos estructurados, semi-estructurados y no estructurados.
- Procesar grandes volúmenes de información en batch y en near-real-time.
- Realizar consultas analíticas avanzadas que generen conocimiento útil para la toma de decisiones municipales.

La arquitectura deberá integrar:

- **PostgreSQL** → datos estructurados y relacionales.
- **MongoDB** → datos semiestructurados/documentales.
- **Apache Hadoop** → almacenamiento y procesamiento distribuido de grandes volúmenes.
- **Apache Spark** → procesamiento y análisis distribuido de datos.

El proyecto no debe limitarse a "instalar las tecnologías". El objetivo principal es que el estudiante **demuestre mediante consultas, procesamiento y análisis cuándo y por qué utilizar cada tecnología**.

2. Objetivo General

Diseñar e implementar una arquitectura integral de gestión y análisis de datos urbanos basada en:

- PostgreSQL
- MongoDB
- Apache Hadoop
- Apache Spark

que permita almacenar, procesar y analizar grandes volúmenes de datos provenientes de sensores IoT y sistemas urbanos inteligentes, facilitando la generación de conocimiento para la toma de decisiones mediante consultas y procesos de análisis las capacidades, diferencias y dominios de aplicación de cada tecnología.

3. Objetivos Específicos

Al finalizar el proyecto, el estudiante deberá ser capaz de:

1. Modelar e implementar el esquema de datos en PostgreSQL y MongoDB.
2. Diseñar el flujo de ingestión y almacenamiento de datos crudos en HDFS.
3. Implementar pipelines de procesamiento con Spark (batch y, opcionalmente, Structured Streaming).
4. Realizar y documentar consultas representativas sobre cada tecnología que demuestren dominio de:
    - Consultas SQL avanzadas, índices, vistas materializadas, particionamiento y optimización en PostgreSQL.
    - Consultas de agregación, índices, schema design y operaciones CRUD/aggregation pipeline en MongoDB.
    - Operaciones MapReduce o consultas Hive/Impala sobre HDFS.
    - Transformaciones, joins, window functions, optimizaciones y consultas Spark SQL / DataFrame API.
5. Integrar los cuatro sistemas de forma coherente (ej.: datos limpios de Spark → PostgreSQL; datos de sensores → MongoDB → HDFS → Spark).
6. Generar al menos un conjunto de indicadores/dashboards o reportes que apoyen la toma de decisiones urbanas (ej.: congestión, calidad del aire, consumo energético, ocupación de estacionamientos).

SmartCity Analytics: Arquitectura Moderna de Datos Urbanos

Sensores IoT (Fuentes)

Movilidad:
Tráfico y Transporte
(Proviene de MongoDB)
Eventos IoT y JSON

Ambiente:
Clima y Calidad del Aire
(Proviene de PostgreSQL)
Datos estructurados

Energía:
Consumo y Distribución
(Proviene de Hadoop/Spark)
Almacenamiento histórico masivo

El Ecosistema de Almacenamiento y Procesamiento

PostgreSQL
Especialización Tecnológica:
Datos estructurados
Integridad relacional y datos maestros
Escalamiento:
Mínimo de 100k registros

MongoDB
Especialización Tecnológica:
Semi-estructurados
Flexibilidad para eventos IoT y JSON
Escalamiento:
Mínimo de 500k registros

Hadoop/Spark
Especialización Tecnológica:
Almacenamiento histórico masivo
Almacenamiento y analítica distribuida
Escalamiento:
Mínimo de 1M registros

Procesamiento Distribuido con Spark

Motor principal para limpieza, transformaciones complejas y generación de indicadores urbanos en tiempo real

¿Cuándo elegir cada herramienta?
(Matriz de Selección Tecnológica)

PostgreSQL	MongoDB	Hadoop/Spark
Volumen bajo	Volumen medio	Volumen masivo (Big Data)
Esquema rígido	Alta velocidad	Analítica distribuida
	Esquema flexible	

Interfaz/Dashboard (Toma de decisiones)



De Sensores a Decisiones: Ingesta de datos de tráfico, clima y energía para alimentar dashboards analíticos estratégicos

Gemini Notebook

4. Descripción del Problema

La municipalidad de una Ciudad Inteligente ha desplegado miles de sensores distribuidos en distintas zonas urbanas.

Estos sensores generan información relacionada con:

- Calidad del aire.
- Condiciones climáticas.
- Tráfico vehicular.
- Transporte público.
- Consumo energético.
- Ocupación de estacionamientos.
- Eventos urbanos.
- Niveles de ruido.
- Flujo peatonal.

La información debe ser almacenada y posteriormente utilizada para responder preguntas estratégicas de la ciudad.



Estos sistemas generan datos con diferentes características:

- datos estructurados;
- datos semiestructurados;
- grandes volúmenes;
- datos generados continuamente;
- datos históricos;
- datos con diferentes velocidades de generación;
- datos provenientes de múltiples dispositivos.

El reto consiste en determinar:
¿Dónde almacenar cada tipo de información y cómo procesarla para obtener conocimiento útil para la ciudad?
La ciudad ficticia **SmartCity** desea implementar una plataforma para analizar diferentes aspectos de su funcionamiento. Se dispone de sensores y sistemas que generan información sobre:

Movilidad

- vehículos;
- bicicletas;
- motocicletas;
- tráfico;
- velocidad promedio;
- congestionamiento.

Ambiente

- temperatura;
- humedad;
- calidad del aire;
- ruido;
- contaminación.

Energía

- consumo eléctrico;
- generación;
- demanda;
- utilización de infraestructura.

Espacios públicos

- ocupación;
- cantidad de personas;
- utilización de parques;
- estacionamientos.

Transporte público

- buses;
- rutas;
- paradas;
- pasajeros;
- tiempos de llegada.

<u>El grupo debe integrar como mínimo dos fuentes de datos diferentes</u>.

Por ejemplo:

- **Movilidad**

Datos de tráfico + Datos meteorológicos → PostgreSQL, MongoDB, HDFS → Spark → Análisis de congestión → Dashboard

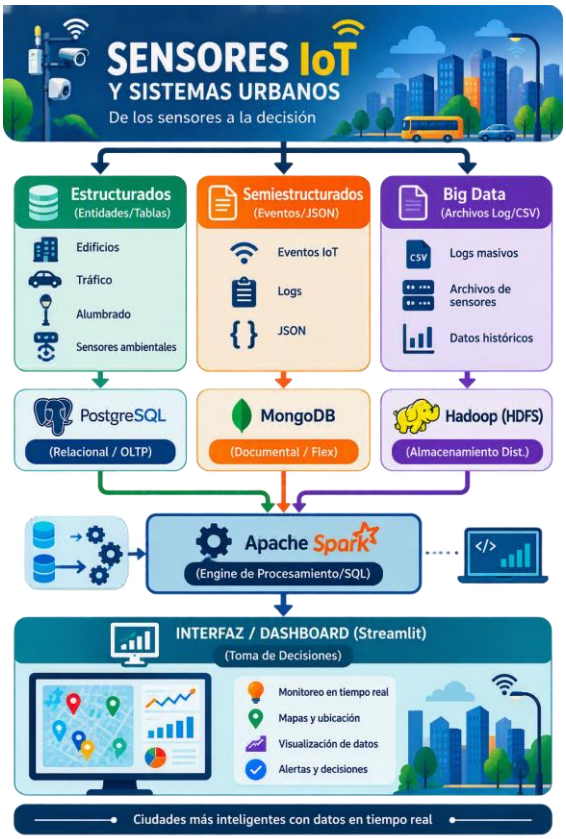
- **Energía**

Consumo eléctrico + Temperatura + Hora/día → Análisis Spark → Predicción de picos → Dashboard

- **Ambiente**

Sensores ambientales + Datos meteorológicos + Ubicación geográfica → MongoDB + PostgreSQL → Hadoop → Spark → Identificación de zonas críticas

Obviamente todo alrededor de IoT.



Tamaño mínimo del dataset

No es obligatorio manejar varios GB porque puede ser difícil para equipos con computadoras con recursos limitados. Propuesta:

Nivel	Volumen	Evaluación
Mínimo	100 MB	Aceptable
Recomendado	500 MB – 1 GB	Bueno
Avanzado	1 – 5 GB	Excelente
Muy Avanzado	**> 5 GB**	Sobresaliente

Pero hay una condición importante:

El tamaño por sí solo no demuestra Big Data.

Un archivo de 5 GB almacenado en una computadora y procesado con Python no demuestra dominio de Hadoop/Spark.

Lo que interesa es demostrar **por qué se necesita procesamiento distribuido y cómo se comporta la arquitectura.**

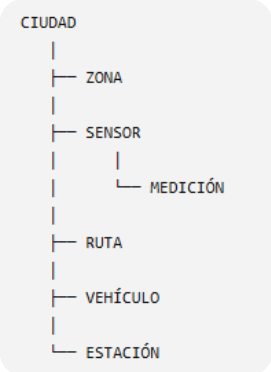
5. Tecnologías obligatorias y rol de cada una

Tecnología	Rol principal en el proyecto
PostgreSQL	Almacenamiento relacional de datos maestros, datos de referencia, datos transaccionales limpios y resultados agregados de alta calidad.
MongoDB	Almacenamiento de documentos JSON/BSON provenientes de sensores IoT (flexibilidad de esquema, series temporales, eventos).
Apache Hadoop (HDFS + MapReduce o Hive)	Almacenamiento distribuido de datos históricos masivos (raw e intermedios) y procesamiento batch de grandes volúmenes.
Apache Spark	Motor de procesamiento distribuido (batch y streaming) para limpieza, transformación, agregaciones complejas, machine learning básico y consultas analíticas sobre los datos almacenados en HDFS y/o MongoDB.

a. Uso de PostgreSQL

PostgreSQL deberá almacenar información altamente estructurada. PostgreSQL deberá utilizarse para representar la información **estructurada y relacional**.

Los estudiantes deberán diseñar un modelo que contenga, como mínimo:



Entidades mínimas

- Sensores
  - id_sensor
  - nombre
  - tipo_sensor
  - fecha_instalacion
  - estado
- Ubicaciones
  - id_ubicacion
  - nombre_zona
  - distrito
  - coordenadas
- Categorías de Sensores
  - id_categoria
  - nombre_categoria

- Usuarios del Sistema
    - id_usuario
    - nombre
    - correo
- Organismos Responsables
    - id_organismo
    - nombre
    - responsable

Requisitos

Los estudiantes deberán:

- Diseñar el modelo entidad-relación.
- Aplicar normalización hasta 3FN.
- Definir llaves primarias.
- Definir llaves foráneas.
- Crear índices apropiados.
- Justificar decisiones de diseño.

Consultas requeridas en PostgreSQL

El proyecto deberá demostrar, como mínimo:

- **Consultas básicas** SELECT – WHERE - ORDER BY - DISTINCT
- **Consultas relacionales** JOIN - INNER JOIN - LEFT JOIN
- **Consultas analíticas** GROUP BY - HAVING – COUNT - SUM – AVG - MIN - MAX
- **Consultas avanzadas**
    - Se deberá incluir al menos:
        - subconsultas;
        - CTE;
        - funciones de ventana;
        - consultas sobre fechas;
        - consultas utilizando índices.

Ejemplos de preguntas

El sistema deberá poder responder:

- ¿Cuál zona registra mayor cantidad de vehículos?
- ¿Cuál es la temperatura promedio por zona?
- ¿Qué sensores han dejado de enviar información?
- ¿Cuál es la hora de mayor congestión?
- ¿Qué rutas presentan mayor utilización?
- ¿Cuál zona presenta mayor contaminación?

Consultas Obligatorias en PostgreSQL

Implementar al menos las siguientes consultas:

1. Sensores por tipo.
2. Sensores por ubicación.
3. Cantidad de sensores activos e inactivos.
4. Número de sensores por organismo responsable.
5. Ubicaciones con mayor cantidad de sensores.
6. Sensores instalados por año.
7. Sensores sin actividad registrada.
8. Ranking de zonas monitoreadas.
9. Consulta utilizando vistas.
10. Consulta utilizando procedimientos almacenados o funciones.

b. Uso de MongoDB

MongoDB se utilizará para almacenar información semiestructurada y documental generada por los sensores. Ejemplo de documento:

```
{
    "sensor": "S-001",
    "tipo": "calidad_aire",
    "ubicacion": {
        "zona": "Centro",
        "lat": 9.9321,
        "lon": -84.0845
    },
    "medicion": {
        "co2": 425,
        "pm25": 18.5,
        "temperatura": 27.2
    },
    "timestamp": "2026-09-05T14:35:00"
}
```

El estudiante deberá demostrar por qué este tipo de información puede ser apropiado para un modelo documental.

```
{
    "sensor_id":"S2001",
    "timestamp":"2026-06-20T14:00:00",
    "tipo":"calidad_aire",
    "mediciones":{
        "pm25":18.6,
        "pm10":25.1,
        "co":0.85,
        "no2":12.3
    },
    "ubicacion":"Zona Norte"
}
```

El estudiante deberá demostrar por qué este tipo de información puede ser apropiado para un **modelo NoSQL**.

Requisitos

El modelo documental deberá almacenar:

- Lecturas de sensores.
- Eventos detectados.
- Alertas automáticas.
- Incidentes urbanos.
- Información dinámica no estructurada.

Preguntas que deberán resolverse con MongoDB

Por ejemplo:

- ¿Cuál es la temperatura promedio por zona?
- ¿Qué sensores registraron valores superiores al límite establecido?
- ¿Cuántas mediciones produjo cada sensor?
- ¿Qué zonas presentan mayor concentración de contaminación?
- ¿Cuál es la evolución de una variable durante un período?

La intención es demostrar que MongoDB no se utiliza simplemente porque "almacena JSON", sino porque permite trabajar de manera flexible con **documentos y estructuras semiestructuradas**.

Consultas requeridas en MongoDB

Deberá utilizarse para demostrar el problema de **Big Data y almacenamiento distribuido**. Los estudiantes deberán trabajar con un conjunto de datos suficientemente grande para justificar su uso.

Deberán demostrarse operaciones:

- *insertOne*
- *insertMany*
- *find*
- *findOne*
- *updateOne*
- *updateMany*
- *deleteOne*

Además:

- *filtros;*
- *operadores;*
- *proyecciones;*
- *ordenamiento;*
- *agregaciones;*
- *agrupaciones;*
- *índices.*

Se deberá utilizar obligatoriamente el **Aggregation Framework**.

Ejemplo conceptual:

```javascript
db.mediciones.aggregate([
    {
        $group: {
            _id: "$ubicacion.zona",
            temperaturaPromedio: {
                $avg: "$medicion.temperatura"
            }
        }
    }
])
```

Consultas Obligatorias en MongoDB

Por ejemplo:

- ¿Cuál es la temperatura promedio por zona?
- ¿Qué sensores registraron valores superiores al límite establecido?
- ¿Cuántas mediciones produjo cada sensor?
- ¿Qué zonas presentan mayor concentración de contaminación?
- ¿Cuál es la evolución de una variable durante un período?
- La intención es demostrar que MongoDB no se utiliza simplemente porque "almacena JSON", sino porque permite trabajar de manera flexible con **documentos y estructuras semiestructuradas**.

Implementar al menos:

1. Lecturas de un sensor específico.
2. Lecturas en un rango de fechas.
3. Valores máximos y mínimos registrados.
4. Eventos críticos detectados.
5. Alertas emitidas por zona geográfica.
6. Promedio de mediciones por zona.
7. Promedio por tipo de sensor.
8. Incidentes registrados por categoría.
9. Consulta utilizando agregaciones.
10. Consulta utilizando filtros múltiples.

c. Dominio de Apache Hadoop

Utilizar Hadoop como repositorio distribuido de datos históricos. Se deberá:

- Instalar Hadoop en modo pseudo-distribuido o mediante Docker.
- Configurar HDFS.
- Diseñar la estructura de almacenamiento.

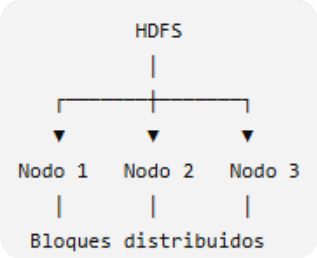
Hadoop deberá utilizarse para demostrar el problema de **Big Data y almacenamiento distribuido**. Se deberá trabajar con un conjunto de datos suficientemente grande para justificar el uso de Hadoop.

Hadoop Distributed File System

Datos por almacenar en HDFS

- Datos históricos masivos.
- Exportaciones JSON.
- Exportaciones CSV.
- Registros de sensores.
- Eventos históricos.

Los estudiantes deberán demostrar conceptualmente:



Deberán explicar:

- almacenamiento distribuido;
- bloques;
- replicación;
- tolerancia a fallos;
- escalabilidad;
- procesamiento cercano a los datos.

Operaciones requeridas en Hadoop

Se deberá demostrar operaciones sobre HDFS, tales como:

- hdfs dfs -mkdir
- hdfs dfs -put
- hdfs dfs -ls
- hdfs dfs -cat

- hdfs dfs -get
- hdfs dfs -rm

Además, deberán explicar *qué sucede cuando un archivo se distribuye entre diferentes nodos*.

Preguntas para Hadoop

El estudiante deberá demostrar procesamiento de grandes volúmenes para preguntas como:

- ¿Cuántas mediciones existen por sensor?
- ¿Cuál sensor produjo mayor cantidad de datos?
- ¿Cuál es la temperatura promedio histórica?
- ¿Cuál es la cantidad de eventos por zona?
- ¿Cuál es el comportamiento de los sensores durante un período determinado?

La intención es que el estudiante pueda explicar:
¿Por qué este problema podría justificar un sistema distribuido?

Cada equipo deberá explicar:

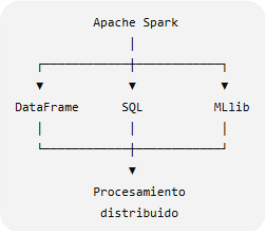
- ¿Por qué utilizar Hadoop?
- ¿Qué ventajas ofrece respecto al almacenamiento tradicional?
- ¿Cómo se organizan los datos?
- ¿Cómo se garantiza escalabilidad?
- ¿Cuántas mediciones existen por sensor?
- ¿Cuál sensor produjo mayor cantidad de datos?
- ¿Cuál es la temperatura promedio histórica?
- ¿Cuál es la cantidad de eventos por zona?
- ¿Cuál es el comportamiento de los sensores durante un período determinado?
- La intención es que el estudiante pueda explicar:
- **¿Por qué este problema podría justificar un sistema distribuido?**

d. Dominio de Apache Spark

Spark será utilizado para el **procesamiento distribuido y análisis de grandes volúmenes de datos**. Spark se utilizará para:

- Procesamiento masivo.
- Generación de estadísticas.
- Análisis históricos.
- Detección de patrones.
- Construcción de indicadores urbanos.

La arquitectura conceptual será:



Se deberá utilizar preferiblemente **Spark SQL y DataFrames**.

Tecnologías permitidas

- Spark SQL
- DataFrames
- Dataset API

Análisis requerido-obligatorios con Spark

Se deberá implementar al menos:

- **Análisis descriptivo:** Ejemplo: Sensores más activos.
- **Análisis temporal:** Ejemplo: Evolución mensual de contaminantes. ¿Cómo cambia la temperatura durante el día?
- **Análisis correlacional:** Ejemplo: Relación entre tráfico y contaminación.
- **Análisis predictivo o de tendencias:** Ejemplo: Predicción de crecimiento del flujo vehicular.
- **Análisis espacial:** ¿Qué zonas presentan mayor contaminación?
- **Análisis de movilidad:** ¿Cuáles son las horas de mayor tráfico?
- **Detección de anomalías:** ¿Qué sensores presentan valores anormales?
- **Comparación:** ¿Cómo se comportan diferentes zonas?

Consultas con Spark SQL

Se deberá cargar información y realizar consultas como:

```sql
SELECT zona,
       AVG(valor) AS promedio
FROM mediciones
GROUP BY zona
ORDER BY promedio DESC;
```

También deberán realizar:

- filtros;
- agrupaciones;
- joins;
- agregaciones;
- ordenamientos;
- consultas temporales.

Consultas Obligatorias con Apache Spark

Implementar al menos:

1. Sensores que generan más información.
2. Distribución geográfica de eventos.
3. Evolución temporal de indicadores urbanos.
4. Correlación entre variables.
5. Ranking de zonas con mayor actividad.
6. Tendencias por período.
7. Detección de anomalías.
8. Agrupamiento por categorías.
9. Comparación histórica entre años.
10. Dashboard analítico o reporte consolidado.

6. La parte más importante: comparación tecnológica

Se deberá demostrar que las tecnologías **no son intercambiables sin considerar el problema**.

Se deberá elaborar una matriz:

Modelo	Relacional	Documental	Distribuido	Procesamiento distribuido
Datos estructurados	Excelente	Bueno	Bueno	Excelente
Datos semiestructurados	Bueno	Excelente	Excelente	Excelente
Relaciones complejas	Excelente	Limitado/alternativo	No es su propósito principal	Excelente
Grandes volúmenes	Excelente	Excelente	Excelente	Excelente
Consultas SQL	Sí	No tradicionalmente	No directamente	Sí
Procesamiento distribuido	Limitado	Escalable horizontalmente	Sí	Sí
Analítica	Excelente	Buena	Básica/infraestructura	Excelente
Integridad relacional	Excelente	No relacional	No es su objetivo	No es su objetivo principal

El estudiante deberá **justificar cada decisión**, no simplemente llenar la tabla.

Todo el proyecto deberá girar alrededor de esta pregunta:

¿Qué tecnología es la más apropiada para cada tipo de dato y cada necesidad de procesamiento dentro de una ciudad inteligente?

Y deberán demostrar la respuesta mediante experimentación.

7. Requerimientos Mínimos de Interfaz Gráfica

El objetivo de la interfaz gráfica es demostrar la integración funcional de PostgreSQL, MongoDB, Hadoop y Spark. No se evaluará el diseño visual ni el uso de tecnologías avanzadas de desarrollo web, sino la capacidad de interactuar con las distintas fuentes de información.

Se recomienda especialmente **Streamlit** debido a su facilidad de uso y rápida integración con PostgreSQL, MongoDB y Spark. Permite construir rápidamente:

- dashboards
- tablas
- gráficos
- filtros
- mapas
- consultas
- interfaces en Python

Y además combina naturalmente con Spark y herramientas de análisis de datos. Como segunda opción si no es posible la anterior, y también muy recomendada, es el uso de PowerBI.

Estructura mínima exigida en la interfaz de Dashboard

Componente	Especificación Técnica Mínima
Vista Principal (Dashboard)	Una sola pantalla o panel centralizado para evitar dispersión en navegación.
Métricas Clave (KPIs)	Se deben presentar los siguientes indicadores. • Movilidad ◦ Vehículos detectados. ◦ Velocidad promedio. ◦ Congestión promedio. ◦ Accidentes/incidentes. ◦ Tiempo promedio de recorrido. • Ambiente ◦ Temperatura global promedio. ◦ PM2.5 promedio. ◦ CO_2. ◦ Humedad. ◦ Número de alertas ambientales. • Energía ◦ Consumo total. ◦ Consumo promedio por zona. ◦ Consumo por hora. ◦ Picos de consumo. ◦ Variación respecto al período anterior. • IoT ◦ Número de sensores. ◦ Mediciones recibidas. ◦ Sensores activos/inactivos. ◦ Alertas generadas. ◦ Datos por minuto/hora/día. ◦ Total de lecturas registradas por segundo Acá se deben presentar las consultas mencionadas para cada una de las tecnologías, que se mencionó anteriormente, como **consultas obligatorias**. Se debe poder seleccionar la tecnología (postgres, Mongo, Hadoop, Spark) y poder realizar las consultas indicadas.
Gráficos Analíticos	Mínimo 4 visualizaciones distribuidas: 1. Serie Temporal: Evolución temporal de las variables. 2. Comparativa por Zonas: Barras o columnas para evaluar sectores urbanos. 3. Distribución Proporcional: Donas o sectores por tipo de sensor o vehículo. 4. Resultado Analítico de Spark: Resultado proveniente de procesamiento masivo (ej. detecciones de patrones o agrupaciones masivas).

	Este es particularmente importante. Debe ser un resultado generado mediante Spark, no simplemente otro gráfico de PostgreSQL. Ejemplo: "Las tres zonas con mayor incremento de contaminación durante las horas pico." Así se demuestra que la interfaz no solamente muestra datos, sino que presenta conocimiento obtenido mediante procesamiento.
Filtros Dinámicos	Mínimo los siguientes controles interactivos. La interfaz debe permitir cambiar dinámicamente al menos: • Rango de fechas o perfiles horarios (Períodos) • Zona o sector de la ciudad • Tipo de medición o dispositivo. Y sería recomendable agregar: 1. Sensor 2. Tipo de evento 3. Fuente 4. Rango de valores Esto permite validar que la interfaz está realmente conectada con los datos.
Tabla de Detalle	Un componente tabular para explorar los datos procesados con paginación o búsqueda.
Mapa Espacial	Representación en mapa con puntos o capas de calor para datasets que incluyan coordenadas de geolocalización.
Etiquetado de Origen	Cada componente gráfico debe incluir una etiqueta clara que indique de qué tecnología provinieron los datos (PostgreSQL, MongoDB, HDFS o Spark SQL).

Aunque no se solicita una interfaz compleja, si debe ser agradable, y poder navegar a los componentes anteriormente mencionados de una manera óptima, y cada pantalla conectarse con la plataforma correspondiente de acuerdo con lo solicitado.

NO es necesario

Para mantener el proyecto enfocado, no es requerido de forma obligatoria:

❌ Login complejo
❌ Gestión de usuarios
❌ Administración de perfiles
❌ Diseño responsive avanzado
❌ Aplicación móvil
❌ Microservicios
❌ API REST completa
❌ Animaciones avanzadas
❌ Diseño gráfico profesional
❌ CRUD completo de todas las entidades

Lo anterior puede ser opcional. El proyecto evalúa principalmente:

Datos + arquitectura + almacenamiento + procesamiento + consultas + análisis.

8. Entregables

Dentro de la documentación a entregar se debe incluir:

a) **Documento técnico:** Justificación de la arquitectura, diagramas E-R y documentales, matriz comparativa de rendimiento entre motores.

Debe contener:

- descripción del problema;
- fuentes de datos;
- arquitectura;
- justificación tecnológica;
- modelo relacional;
- modelo documental;
- estrategia Big Data;
- procesamiento;
- resultados;
- conclusiones.

Hay que recordar que se **requiere fuerte explicación** de cada decisión tomada en el proyecto.

b) **Código Fuente:** Scripts SQL (DDL y DML), Scripts BSON/MongoDB, Comandos HDFS, Notebooks/Scripts de PySpark y Código de la Interfaz (Streamlit).

i. **Base PostgreSQL**

Incluyendo:

- DDL;
- restricciones;
- relaciones;
- índices;
- consultas.

ii. **Base MongoDB**

Incluyendo:

- documentos;
- consultas;
- actualizaciones;
- agregaciones;
- índices.

iii. **Implementación Hadoop**

Incluyendo:

- carga de datos;
- estructura HDFS;
- operaciones;
- procesamiento.

iv. **Implementación Spark**

Incluyendo:

- carga de datos (ETLs);
- DataFrames;
- Spark SQL;
- transformaciones;
- acciones;
- análisis.

v. **Dashboard o visualización**

Deberá mostrar algunos resultados obtenidos.

.

c) **Defensa Demostrativa:** Presentación ejecutada en vivo realizando consultas directas sobre cada motor.

La presentación deberá ser principalmente **demostrativa**, no una exposición teórica. Durante la defensa final, la aplicación deberá demostrar:

- ✅ Conexión a PostgreSQL.
- ✅ Conexión a MongoDB.
- ✅ Consulta de información almacenada en HDFS o datos cargados desde Hadoop.
- ✅ Procesamiento mediante Spark.
- ✅ Visualización de resultados.

Cada grupo deberá demostrar:



El profesor podrá realizar preguntas como:

- ¿Por qué almacenaron esta información en PostgreSQL?
- ¿Por qué utilizar MongoDB aquí?
- ¿Qué problema resuelve Hadoop?
- ¿Por qué Spark y no realizar el procesamiento directamente en PostgreSQL?
- ¿Qué sucede si aumenta 100 veces el volumen de datos?
- ¿Cuál tecnología utilizarían si los datos fueran altamente variables?
- ¿Dónde está realmente el valor de utilizar las cuatro tecnologías?

Se recomienda llegar preparados con las posibles respuestas a esas preguntas.

9. Reto adicional

Para grupos que quieran alcanzar un nivel superior, se puede incorporar:

Streaming

Simular la llegada continua de sensores:

```
IoT
 ↓
Eventos
 ↓
Spark Streaming
 ↓
Procesamiento
 ↓
Alerta
```

Por ejemplo:

- Si la contaminación de una zona supera determinado umbral, generar una alerta.

O:

- Si la cantidad de vehículos supera un límite, identificar congestión.

Esto permite introducir el concepto de **procesamiento batch vs. streaming**.

Resultado Final

Al finalizar, el estudiante debería ser capaz de explicar y demostrar algo mucho más importante que el funcionamiento individual de cuatro herramientas:

Los sistemas modernos de datos requieren seleccionar diferentes tecnologías dependiendo de la naturaleza de los datos, el volumen, la velocidad de generación, la estructura, las necesidades de consulta y los objetivos de análisis.

Frase central del proyecto

"El objetivo no es utilizar cuatro tecnologías; es demostrar por qué una arquitectura moderna de datos puede necesitar diferentes tecnologías para almacenar, procesar y analizar diferentes tipos de información."

10. Fuentes de Datos Recomendadas

Se recomienda que los grupos utilicen una o varias de las siguientes fuentes para construir el ecosistema de datos de la Ciudad Inteligente. Se recomienda usar al menos 2 repositorios distintos para sus datos, y estos **deberán cumplir con los requerimientos de cada una de las consultas que cada plataforma** requieren según se indicó anteriormente, con consultas obligatorias. Podrán utilizar datos abiertos provenientes de los siguientes repositorios:

- **OpenAQ**

Datos reales de calidad del aire a nivel mundial. Incluye API abierta y millones de registros históricos. [openaq.org], [docs.openaq.org]
https://openaq.org/

- **CityPulse Smart City Datasets**

  Repositorio académico orientado a ciudades inteligentes con información de:
  - Tráfico
  - Clima
  - Contaminación
  - Estacionamientos
  - Eventos urbanos

[CityPulse...s - Surrey]
http://iot.ee.surrey.ac.uk:8080/datasets.html

- **Smart City Traffic Flow Prediction Dataset**

Datos de movilidad urbana y tráfico vehicular.
[kaggle.com]
https://www.kaggle.com/

- **Smart Data Models**

Repositorio de modelos de datos estandarizados para ciudades inteligentes.
Se recomienda utilizarlo como referencia para el diseño conceptual.
[fiware.org], [github.com]
https://smartdatamodels.org/

- **IoTDataset**

Colección de datasets IoT y sensores urbanos.
[iotdataset.com]
https://iotdataset.com/

- **FIWARE Smart Data Models**

No es un dataset sino un estándar internacional de modelado de datos para ciudades inteligentes.
Incluye modelos para:

- Transporte.
- Medio ambiente.
- Energía.
- Sensores.
- Gestión urbana.

Sitios:
https://smartdatamodels.org/
https://github.com/smart-data-models/
[fiware.org], [github.com]

- **Kaggle**

Es probablemente una de las mejores fuentes para el proyecto. Actualmente muestra **cientos de miles de datasets públicos**, incluyendo conjuntos desde MB hasta varios GB. Por ejemplo, aparecen datasets de Bitcoin, imágenes de 8 GB y un dataset de arXiv de aproximadamente 2 GB.

Ideal para:

- IoT
- transporte
- ciudades
- energía
- sensores
- ventas
- imágenes
- redes
- series temporales

- **UCI Machine Learning Repository**

Excelente fuente académica. Actualmente mantiene **689 datasets**.

Hay algunos particularmente interesantes para este proyecto.

Por ejemplo:

Individual Household Electric Power Consumption

Tiene aproximadamente **2.08 millones de registros**, con mediciones de consumo eléctrico a intervalos de un minuto durante casi cuatro años.

También existe **Online Retail II**, con aproximadamente **1.07 millones de instancias**.

Y **Covertype**, con aproximadamente **581.000 instancias**.

- **Data.gov**

Permite acceder a datos públicos del gobierno de Estados Unidos.

Hay información relacionada con:

- transporte
- población
- clima
- criminalidad
- energía
- salud
- geografía
- infraestructura

Esto permite que los estudiantes trabajen con **datos reales**, no solamente datasets preparados para Machine Learning.

- **World Bank Open Data**

Muy recomendable para incorporar variables socioeconómicas:

- población
- PIB
- energía
- educación
- salud
- emisiones
- indicadores económicos

Escalamiento de Datos
Los datasets originales podrán ampliarse mediante generación sintética de datos.

Cada grupo deberá trabajar con un volumen mínimo equivalente a:

Escenario mínimo
- 100.000 registros PostgreSQL
- 500.000 documentos MongoDB
- 1.000.000 registros procesados mediante Spark

Escenario recomendado
- Más de 5 millones de registros totales

Los estudiantes deberán documentar el mecanismo utilizado para generar dichos datos.

Recomendaciones prácticas para el proyecto

1. **Combinar fuentes** (ejemplo muy completo):
    - Sensores IoT de calidad del aire → MongoDB
    - Datos de tráfico de NYC o Europa → HDFS + Spark
    - Datos maestros (ubicaciones de sensores, zonas, calendarios) → PostgreSQL
2. Empezar con un subconjunto (1-5 GB) para desarrollo y luego escalar a decenas de GB para las pruebas de rendimiento.
3. Preferir formatos **Parquet** o **CSV comprimidos** para facilitar la carga en Spark/Hadoop.
4. Documentar siempre la fuente, licencia y proceso de descarga/limpieza.

Con estos requisitos de interfaz (mínimos y realistas) y las fuentes de datos indicadas, el proyecto queda perfectamente alineado con los objetivos del curso y es viable de realizar en un semestre universitario.

11. Documento de Atributos

Objetivo general

- ➔ Debe comentar de qué manera se aplicaron los atributos que posee el curso y su impacto durante la implementación del proyecto. Detalle para cada atributo lo siguiente:
    1. Aplicación del atributo en la solución del proyecto
    2. Impacto del proyecto en la sociedad
    3. Retroalimentación obtenida gracias al proyecto

Favor colocar la información antes solicitada para cada uno de los atributos siguientes:

Los atributos del graduado a desarrollar en este curso son los siguientes:

- ◆ **Análisis de Financiero (AF)**

Aplica el conocimiento y la comprensión de los principios de gestión de ingeniería y la toma de decisiones económicas y los aplica al propio trabajo, como miembro y líder de un equipo, y para gestionar proyectos en entornos multidisciplinarios.

- ◆ **Aprendizaje continuo AC (Medio)**

Capacidad para ejecutar estrategias con el fin de resolver sus necesidades educativas o de aprendizaje, de acuerdo con los objetivos o las intenciones de aprendizaje.

- ➔ Presente la siguiente información:
    1. Documentación generada del proyecto final del Curso
    2. Documento con la información de cada atributo dentro del proyecto. Mínimo un párrafo por atributo.

Matriz de Calificación

Criterio	Pts	Excelente (100%)	Bueno (80%)	Básico (60%)	No Presentado (0%)
1. Cuadro Comparación Tecnológica	7	Comparación completa, análisis crítico y justificación técnica sólida	Comparación adecuada con justificación parcial	Comparación limitada y superficial	No presenta
2. Dashboard (Interfaz)	14	Incluye vista principal, KPIs, gráficos, filtros, tabla, mapa y etiquetado totalmente funcionales	Cumple la mayoría de componentes con pocos errores	Componentes básicos con funcionalidad limitada	No presenta
3. Consultas de cada Tecnología	15	Se puede acceder a las consultas de cada tecnología de forma efectiva	La mayoría de las consultas funcionan adecuadamente	Algunas consultas incompletas o con errores	No presenta
4. Documento Técnico	10	Documento completo, estructurado y bien documentado	Contiene la mayoría de elementos requeridos	Información parcial o poco detallada	No presenta
5. Código Fuente	2	Código limpio, modular, documentado y funcional	Código funcional con pequeños problemas de organización	Código básico con errores o poca documentación	No presenta
6. PostgreSQL (CTEs y Window Functions)	7	Diseño óptimo y uso correcto de CTEs y funciones ventana	Implementa consultas avanzadas con pocos errores	Uso básico de SQL sin explotar funcionalidades avanzadas	No presenta
7. MongoDB (Aggregation Framework)	7	Modelado y agregaciones avanzadas correctamente implementadas	Agregaciones funcionales con limitaciones menores	Consultas básicas y poco uso del framework	No presenta
8. HDFS (Gestión y Almacenamiento)	7	Administra correctamente archivos, directorios y distribución.	Utiliza adecuadamente los comandos principales	Uso limitado de comandos y conceptos	No presenta
9. Apache Spark (DataFrames y SQL)	7	Procesamiento distribuido eficiente usando DataFrames y Spark SQL	Implementación funcional con algunos aspectos por mejorar	Uso básico de Spark	No presenta
10. Interfaz Gráfica Analítica	7	Diseño profesional, KPIs y filtros totalmente funcionales	Interfaz adecuada con ligeros problemas de usabilidad	Diseño básico con funcionalidad limitada	No presenta
11. Defensa Oral y Sustentación Técnica	15	Excelente dominio conceptual y técnico, responde con seguridad	Demuestra conocimiento general del proyecto	Presentación limitada y respuestas incompletas	No presenta
12. Documento de Atributos	2	Define claramente atributos, tipos, fuentes y restricciones	Incluye la mayoría de atributos requeridos	Documento incompleto o poco detallado	No presenta

Cada punto debe tener una correcta y amplia justificación de las decisiones tomadas.